

Recurrent Neural Networks

Exploring RNN
architectures, LSTM
gating
mechanisms, and
encoder-decoder
models

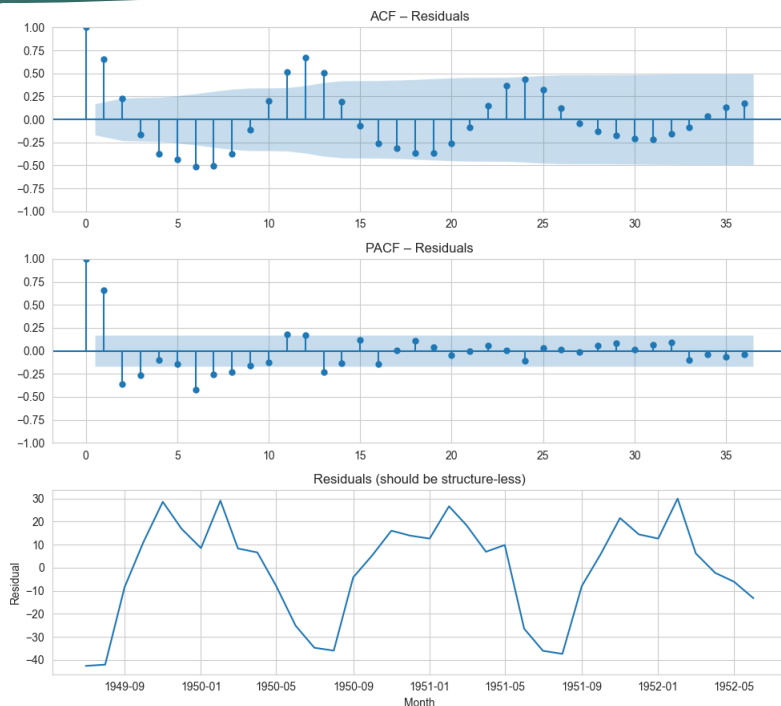
Εργ. Βοηθός: Αντώνης Τζιόρβας
Email: atzio@unipi.gr

Intro to Time Series Analysis (1)– Seasonal Decomposition

- ❖ A time series y_t can be expressed as an additive (multiplicative) decomposition as follows:
 $y_t = T_t + (\times) S_t + (\times) R_t$ where:
 - ❖ The trend captures long-term movement
 - ❖ The seasonal component repeats with a fixed period and
 - ❖ The residual contains the unexplained noise.
- ❖ **Seasonal decomposition** isolates these three parts by applying a MA filter to estimate the trend and then extracting the periodic pattern from the detrended series.
- ❖ **Trend strength** $\max(0, 1 - \left(\frac{\text{Var}(R_t)}{\text{Var}(T_t + R_t)}\right))$ quantifies how much of the total variance is explained by the trend; values close to 1 indicate a dominant trend.
- ❖ **Seasonality strength** $= \max(0, 1 - \left(\frac{\text{Var}(R_t)}{\text{Var}(S_t + R_t)}\right))$ measures the explanatory power of the seasonal pattern in the same way.

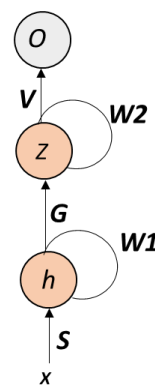
Intro to Time Series Analysis (2) – Lag Plots, ACF/PACF & Unit-Root Tests

- ❖ Lag plot visualizes the relationship between x_t and x_{t-k} ; a strong linear pattern indicates autocorrelation, which is typical for non-stationary series.
 - ❖ Autocorrelation Function (ACF) shows correlation of the series with its own lags; a slowly decaying ACF is a hallmark of a unit-root process.
 - ❖ Partial Autocorrelation Function (PACF) isolates the direct effect of each lag, helping to identify the appropriate AR order for ARIMA modelling.
-
- ❖ Augmented Dickey-Fuller (ADF) test evaluates the null hypothesis “the series has a unit root”.
p-value > 0.05 $\Rightarrow$ fail to reject $\Rightarrow$ series is non-stationary.
 - ❖ KPSS test assesses the opposite null hypothesis “the series is stationary around a deterministic trend”.
p-value < 0.05 $\Rightarrow$ reject $\Rightarrow$ evidence of a unit root.

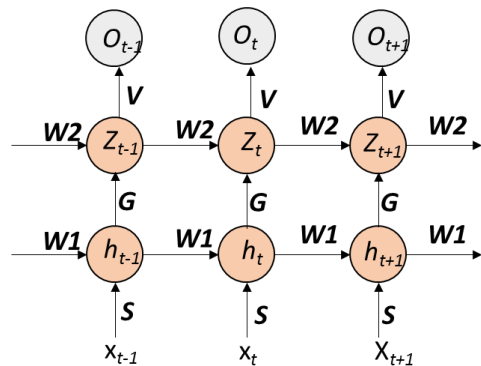


RNN Overview – Why Sequences Need Special Networks

- ❖ Sequences have temporal dependencies that feed-forward nets cannot capture
- ❖ Recurrent connections allow hidden state to **persist** information across timesteps
- ❖ Unfolding an RNN reveals three parameter blocks: input $\rightarrow$ hidden, hidden $\rightarrow$ hidden, hidden $\rightarrow$ output
- ❖ Each block is a shallow affine transform followed by a fixed non-linearity
- ❖ Introducing depth inside these blocks improves representational power



a) 2-layer Recurrent Neural Network (RNN)



b) Unfolded 2-layer Recurrent Neural Network (RNN)

Deep Recurrent Networks – Adding Depth to the Three Blocks

- ❖ Depth can be added to input-to-hidden, hidden-to-hidden, or hidden-to-output transforms
- ❖ A deep MLP replaces the single affine layer, yielding richer feature mappings
- ❖ Deeper recurrences help model complex temporal patterns without changing the recurrence length
- ❖ **Implementation:** stack `Dense` layers before the recurrent cell in Keras

```
model = keras.Sequential([  
    keras.layers.Input(shape=(timesteps, features)),  
    keras.layers.Dense(64, activation="relu"),  
    keras.layers.LSTM(128),  
    keras.layers.Dense("num_classes", activation="softmax"),  
])
```

Challenges of Long-Term Dependencies (1)

❖ Core Issue

- ❖ Gradients propagated through many timesteps tend to **vanish** or **explode**.
- ❖ Each timestep requires a matrix-vector multiplication →
⇒ Long sequences ⇒ **High runtime & memory**
⇒ **Expensive** gradient back-propagation

❖ Underlying Cause

- ❖ Matrix-multiplication-like composition of Jacobians → exponential growth or decay.
- ❖ Even stable RNNs give exponentially smaller influence on distant steps.

❖ Partial Solutions

- ❖ **LSTM/GRU gating** mitigates (but doesn't eliminate) the issue.
- ❖ Proper initialization, gradient clipping, and architecture design are essential.

Challenges of Long-Term Dependencies (2)

	Vanishing Gradients	Exploding Gradients
Cause	Repeated multiplication by weights < 1	Repeated multiplication by weights > 1
Effect	Gradients $\rightarrow 0 \rightarrow$ early timesteps forgotten	Gradients $\rightarrow \infty \rightarrow$ instability
Symptoms	Loss plateaus quickly; only short-term dependencies learned	NaNs, overflow, unstable training
Fixes	Better initialization, LSTM/GRU units	Gradient clipping (by norm or value)

Long Short-Term Memory Cell – Core Gating Mechanism

- ❖ Four main components: **input gate**, **forget gate**, **cell state**, **output gate**.
- ❖ All gates use a **sigmoid** (σ) activation; the **candidate memory** ($\tilde{C}_t$) typically uses **tanh** (keeps candidate values bounded between -1 and 1).
- ❖ **Forget Gate** (top-left σ gate)
 - ❖ Decides **what to forget** from the old memory (C_{t-1}).
 - ❖ Uses x_t and $h_{t-1} \rightarrow$ passes through a sigmoid (σ) $\rightarrow$ output between 0 and 1.
 - ❖ Multiplies with old cell state $\rightarrow$ forgets parts of memory.
- ❖ **Input Gate** (middle σ and tanh gates)

- ❖ Decides **what new info to store**.
- ❖ Sigmoid (σ) gate: decides which values to update.
- ❖ **Tanh** gate: creates candidate values to add to memory.
- ❖ These are combined and added to the cell state.

❖ **Output Gate** (right σ and tanh gates)

- ❖ Decides **what to output** (h_t) based on updated memory.
- ❖ Uses x_t and $h_{t-1} \rightarrow$ passes through a sigmoid $\rightarrow$ multiplies/scales the tanh of the new cell state $\rightarrow$ gives output h_t .

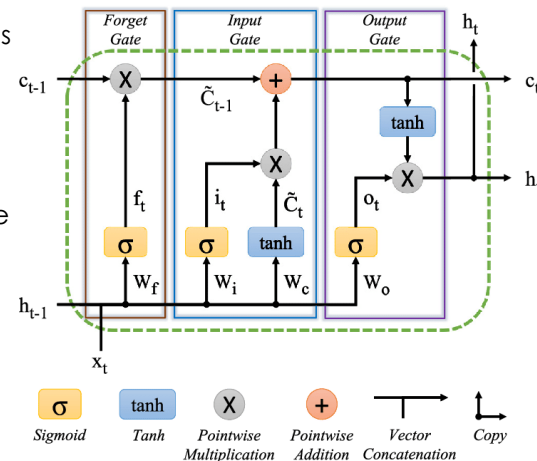
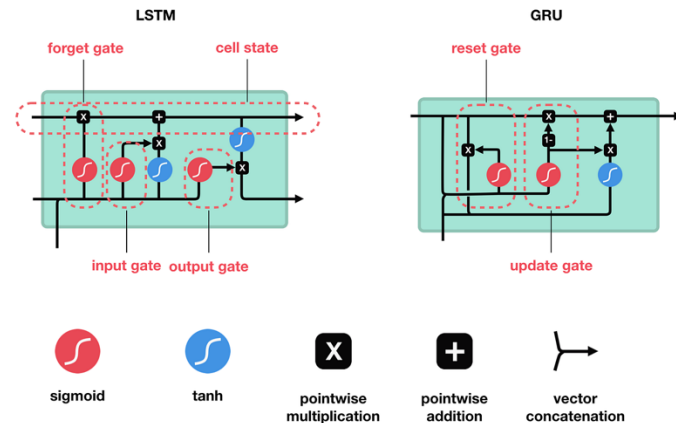


Image Source: [GeeksforGeeks](https://www.geeksforgeeks.org/)

GRU – Simpler Gated Alternative

- ❖ Merges cell & hidden state into a single vector
- ❖ Update gate $z(t)$ – decides how much of the past to keep
- ❖ Reset gate $r(t)$ – controls how to combine new input with past
- ❖ Fewer parameters than LSTM, often faster to train



Building a Simple LSTM Language Model (1)

- ❖ Load a small text corpus and map characters to integer IDs
- ❖ Create training sequences of fixed length (e.g., 40 chars) with next-char targets
- ❖ Use an Embedding layer to convert IDs to dense vectors
- ❖ Add a single LSTM layer to capture temporal context

```
text = open("path/to/data").read()
chars = sorted(set(text))
char2idx = {c: i for i, c in enumerate(chars)}
seq_len = 40
X = []
Y = []

for i in range(0, len(text) - seq_len):
    X.append([char2idx[c] for c in text[i : i + seq_len]])
    Y.append(char2idx[text[i + seq_len]])

X = keras.preprocessing.sequence.pad_sequences(X, padding="post")
Y = keras.utils.to_categorical(Y, num_classes=len(chars))
```

Building a Simple LSTM Language Model (2)

- ❖ Compile the model with categorical cross-entropy and Adam optimizer
- ❖ Train for a few epochs and monitor training loss
- ❖ Generate text by feeding the model's own predictions back as input

```
model = keras.Sequential([
    keras.layers.Embedding(
        input_dim=len(chars),
        output_dim=...,
        input_length=seq_len
    ),
    keras.layers.LSTM(...),
    keras.layers.Dense(len(chars), activation="softmax"),
])
model.compile(loss="...", optimizer="...")
model.fit(X, Y, epochs=5, batch_size=...)
```

Visualizing LSTM Gates

Extract

Extract gate activations by defining a custom ``LSTMCell`` subclass

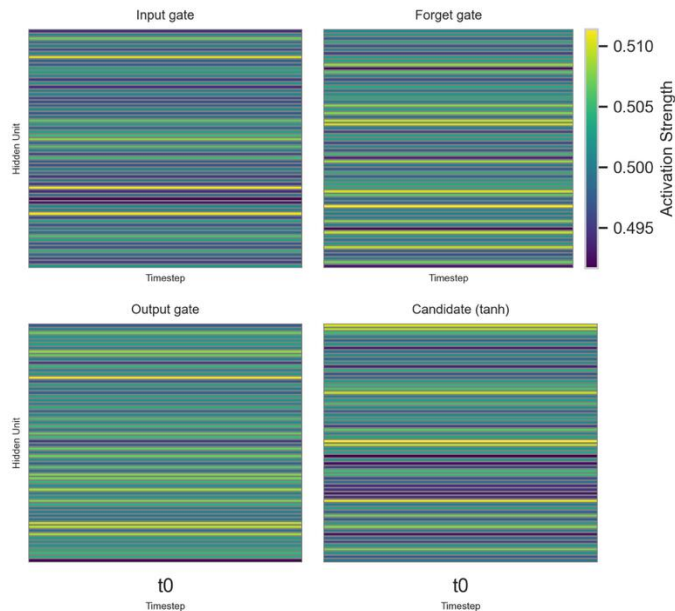
Plot

Plot input, forget, and output gate values over a sample sentence

Observe

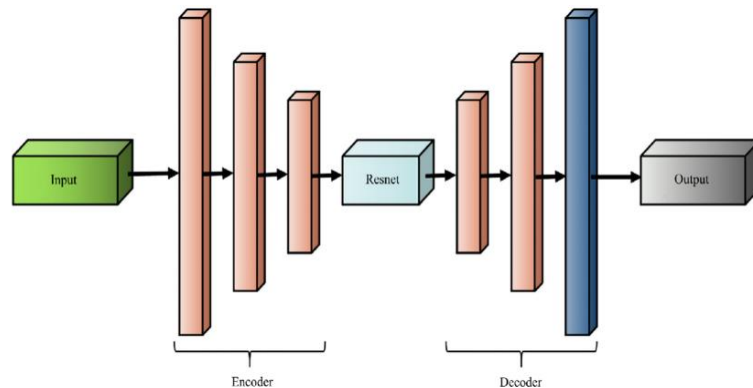
Observe higher forget-gate values at punctuation boundaries

LSTM Gate Activations for the Sentence "to be or not to be"



Encoder-Decoder (Seq2Seq) Architecture

- ❖ Encoder reads the source sequence and compresses it into a context vector (final hidden state)
- ❖ Decoder generates the target sequence step-by-step, conditioned on the context
- ❖ Teacher forcing feeds the ground-truth previous token during training for faster convergence
- ❖ Keras functional API makes wiring encoder and decoder straightforward



Training the Seq2Seq Model

- ❖ Prepare parallel sentence pairs and tokenize both source and target languages
- ❖ **Pad sequences** to uniform length and shift target inputs by one timestep for teacher forcing
- ❖ Fit the model with `batch\_size` and monitor **validation loss**
- ❖ After training, inference uses **only the encoder state** and a loop that feeds the previous prediction back

```
# Shift target for teacher forcing
decoder_input_data = target_sequences[:, :-1]
decoder_target_data = target_sequences[:, 1:]
seq2seq.fit(
    [encoder_input_data, decoder_input_data],
    decoder_target_data,
    batch_size=64,
    epochs=30,
    validation_split=0.2,
)
```

Stacked LSTM – Adding Depth to Encoder/Decoder

❖ Why stack LSTMs?

- ❖ A single recurrent layer can capture short-range patterns,
 - ❖ Real-world sequences contain hierarchical structures
 - ❖ Deeper recurrent stacks can capture hierarchical temporal patterns
-
- ❖ Stack two LSTM layers by setting ``return_sequences=True`` on the first layer
 - ❖ Training may require more epochs and careful regularization

Stacked encoder

```
enc_lstm1 = keras.layers.LSTM(  
    256,  
    return_sequences=True,  
    return_state=True  
)  
  
enc_lstm2 = keras.layers.LSTM(256, return_state=True)  
  
enc_outputs1, h1, c1 = enc_lstm1(enc_inputs)  
_, enc_h, enc_c = enc_lstm2(enc_outputs1)
```

Same stacking applies to decoder

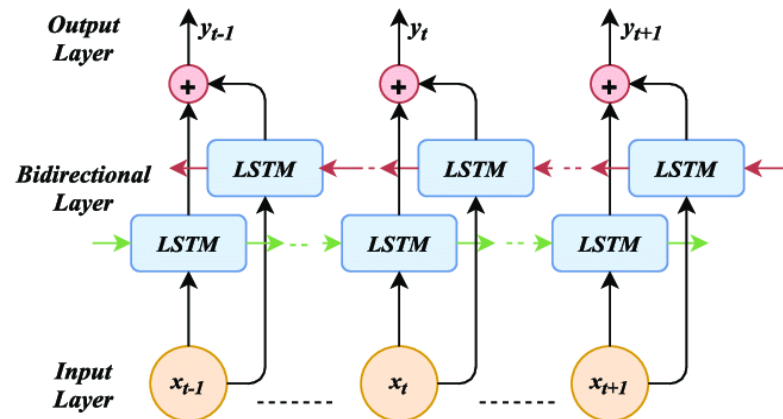
Gradient Clipping – Stabilizing Training

- ❖ Clipping prevents **exploding gradients** that can crash training
- ❖ Keras allows clipping by norm or by value directly in the optimizer
- ❖ **Common Clipping Strategies**
 - ❖ Clip by norm: The gradient vector is scaled so its L2-norm never exceeds the threshold.
 - ❖ Clip by value: Each individual gradient component is limited to the interval $[- \text{value}, \text{value}]$
- ❖ **Use Cases**
 - ❖ Training on long sequences ($\tau > 100$).
 - ❖ Observing sudden loss spikes or *inf/nan* values.
 - ❖ After adding depth (more stacked layers) which increases gradient flow paths.

Bidirectional LSTM – Capturing Past & Future Context

- ❖ A bidirectional layer runs two independent LSTMs on the same input sequence
 - ❖ One processes it forward ($t = 1 \rightarrow \tau$)
 - ❖ The other processes it backwards ($t = \tau \rightarrow 1$)
 - ❖ Their hidden states are aggregated at each timestep, giving access to *both* past and future information.
- ❖ Wrap an LSTM with ``Bidirectional`` to process the sequence forward and backward
- ❖ Useful for tasks where the whole sentence is known beforehand (e.g., POS tagging)
- ❖ **Note:** Output dimension doubles, so downstream layers may need adjustment

```
model = keras.Sequential([  
    keras.layers.Bidirectional([keras.layers.LSTM(128)],  
    keras.layers.Dense("num_classes", activation="softmax"),  
])
```



Regularization – Dropout inside LSTM

- ❖ Randomly zero activations during training, improving generalization
- ❖ Input dropout is applied to the inputs x_t before they enter the LSTM cell.
- ❖ Recurrent dropout is applied to the hidden-to-hidden recurrent connections h_{t-1} inside the cell.
 - ❖ Recurrent connections are a major source of over-fitting on sequential data.
- ❖ Typical values: 0.2 – 0.3 for both arguments work well for medium-sized datasets
- ❖ **Note:** Higher rates may degrade the ability to retain long-range information.

Evaluating Sequence Models

- ❖ **Perplexity**: Directly derived from cross-entropy loss ($\text{perplexity} = \exp(\text{loss})$). Lower perplexity indicates the model assigns higher probability to the true sequence. Ideal for language modelling.
- ❖ **BLEU (Bilingual Evaluation Understudy)**: Calculates n-gram overlap between generated translations and reference sentences. Useful for seq2seq translation (**can be noisy on very short sentences**).
- ❖ **ROUGE / METEOR**: Alternative overlap-based scores that may better capture recall-oriented aspects (e.g., summarization).
- ❖ **Accuracy / F1** at the token level: For classification-style tasks (POS, NER) you can compute token-wise accuracy or token-level F1.